

---

# C 程序设计基础及实验

---

主讲教师: 纪守领

学院: 计算机科学与技术

NESA Lab: <https://nesa.zju.edu.cn/>

Email: [sji@zju.edu.cn](mailto:sji@zju.edu.cn)



## Chapter 12 文件

### 12.1 素数文件

### 12.2 用户信息加密和校验

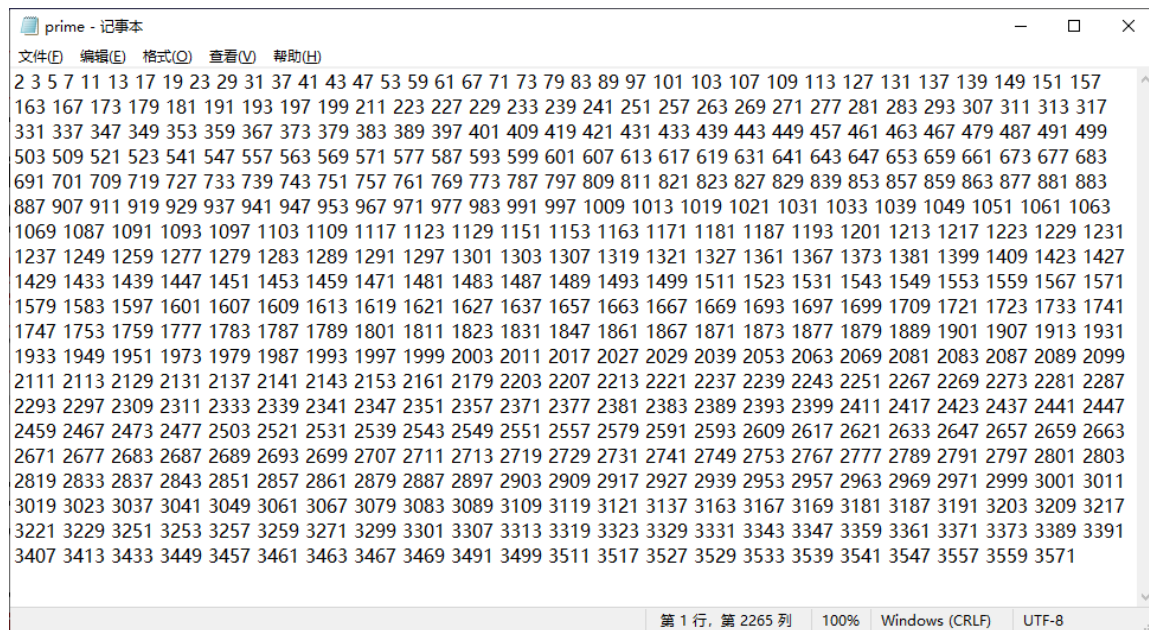
### 12.3 文件综合应用：资金账户管理

## 本章要点

- 什么是文件？C文件是如何存储的？
- 什么是文件缓冲系统？工作原理如何？
- 什么是文本文件和二进制文件？
- 如何打开、关闭文件？
- 如何编写文件读写程序？
- 如何编写程序，实现文件数据管理？

## 12.1 素数文件

□ 例12-1 从2开始依次找出500个素数，将这些素数存入文本文件 prime.txt中



```
prime - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157
163 167 173 179 181 191 193 197 199 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317
331 337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 491 499
503 509 521 523 541 547 557 563 569 571 577 587 593 599 601 607 613 617 619 631 641 643 647 653 659 661 673 677 683
691 701 709 719 727 733 739 743 751 757 761 769 773 787 797 809 811 821 823 827 829 839 853 857 859 863 877 881 883
887 907 911 919 929 937 941 947 953 967 971 977 983 991 997 1009 1013 1019 1021 1031 1033 1039 1049 1051 1061 1063
1069 1087 1091 1093 1097 1103 1109 1117 1123 1129 1151 1153 1163 1171 1181 1187 1193 1201 1213 1217 1223 1229 1231
1237 1249 1259 1277 1279 1283 1289 1291 1297 1301 1303 1307 1319 1321 1327 1361 1367 1373 1381 1399 1409 1423 1427
1429 1433 1439 1447 1451 1453 1459 1471 1481 1483 1487 1489 1493 1499 1511 1523 1531 1543 1549 1553 1559 1567 1571
1579 1583 1597 1601 1607 1609 1613 1619 1621 1627 1637 1657 1663 1667 1669 1693 1697 1699 1709 1721 1723 1733 1741
1747 1753 1759 1777 1783 1787 1789 1801 1811 1823 1831 1847 1861 1867 1871 1873 1877 1879 1889 1901 1907 1913 1931
1933 1949 1951 1973 1979 1987 1993 1997 1999 2003 2011 2017 2027 2029 2039 2053 2063 2069 2081 2083 2087 2089 2099
2111 2113 2129 2131 2137 2141 2143 2153 2161 2179 2203 2207 2213 2221 2237 2239 2243 2251 2267 2269 2273 2281 2287
2293 2297 2309 2311 2333 2339 2341 2347 2351 2357 2371 2377 2381 2383 2389 2393 2399 2411 2417 2423 2437 2441 2447
2459 2467 2473 2477 2503 2521 2531 2539 2543 2549 2551 2557 2579 2591 2593 2609 2617 2621 2633 2647 2657 2659 2663
2671 2677 2683 2687 2689 2693 2699 2707 2711 2713 2719 2729 2731 2741 2749 2753 2767 2777 2789 2791 2797 2801 2803
2819 2833 2837 2843 2851 2857 2861 2879 2887 2897 2903 2909 2917 2927 2939 2953 2957 2963 2969 2971 2999 3001 3011
3019 3023 3037 3041 3049 3061 3067 3079 3083 3089 3109 3119 3121 3137 3163 3167 3169 3181 3187 3191 3203 3209 3217
3221 3229 3251 3253 3257 3259 3271 3299 3301 3307 3313 3319 3323 3329 3331 3343 3347 3359 3361 3371 3373 3389 3391
3407 3413 3433 3449 3457 3461 3463 3467 3469 3491 3499 3511 3517 3527 3529 3533 3539 3541 3547 3557 3559 3571
```

第 1 行, 第 2265 列 100% Windows (CRLF) UTF-8

## 例12-1 源程序

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
int prime(int n); /* 函数声明, 定义略 */
int main(void)
{
    int n = 2, count = 0;
    FILE *fp;
    if((fp = fopen("prime.txt", "w")) == NULL){
        printf("File open error!\n");
        exit(0);
    }
    while(count < 500){
        if(prime(n) != 0){
            count++;
            fprintf(fp, "%d ", n);
        }
        n++;
    }
    if(fclose(fp)){
        printf("Can not close the file!\n");
        exit(0);
    }
    return 0;
}
```

/\* (1) 定义文件指针 \*/  
/\* (2) 打开文件 \*/

/\* (3) 文件处理-写入 \*/

/\* (4) 关闭文件 \*/

## 12.1.2 文件的概念

- **文件**：操作系统中的文件是指驻留在外部介质（如磁盘等）中的一个有序数据集
- **各种类型的文件**
  - **程序文件**：源文件、目标程序、可执行程序
  - **数据文件（输入/输出）**：文本文件、图像文件、声音文件、可执行文件等
- **文件的特点**
  - 数据**永久保存**；数据**长度不定**；数据按**顺序存取**

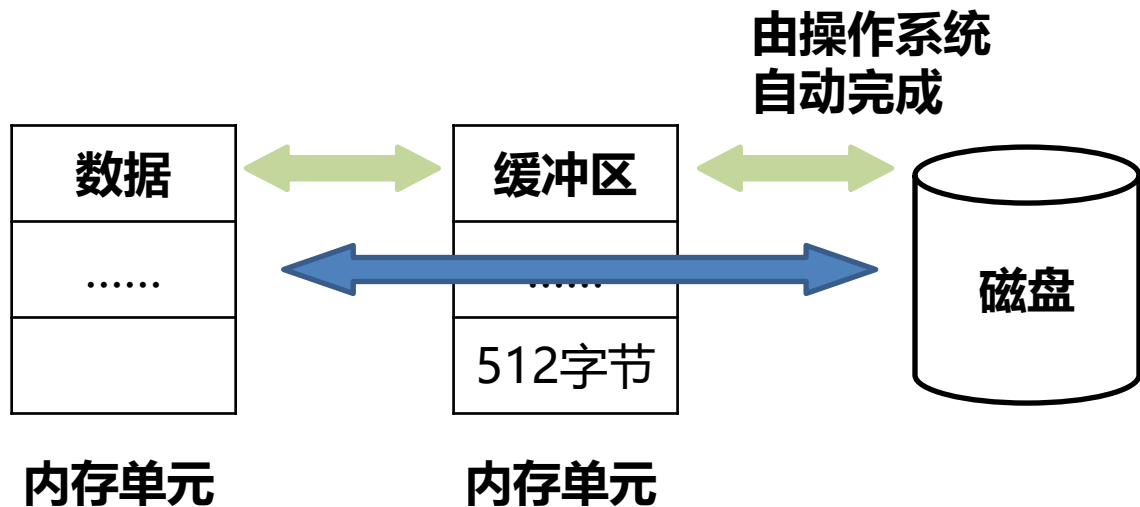
## 12.1.3 文本文件和二进制文件

|    |    |    |    |    |    |       |
|----|----|----|----|----|----|-------|
| 字节 | 字节 | 字节 | 字节 | 字节 | 字节 | ..... |
|----|----|----|----|----|----|-------|

- C语言中的文件是数据流（由一个个的字节数据组成）
- 文件的两种数据形式：
  - ASCII码（文本文件text stream） 字符流
  - 二进制码（二进制文件binary stream） 二进制流
- 二进制文件是直接内存数据以二进制形式保存，例如整数1234
  - 文本文件保存：49 50 51 52（4个字符）
  - 二进制文件保存：04D2（1234的二进制数）

## 12.1.4 缓冲文件系统

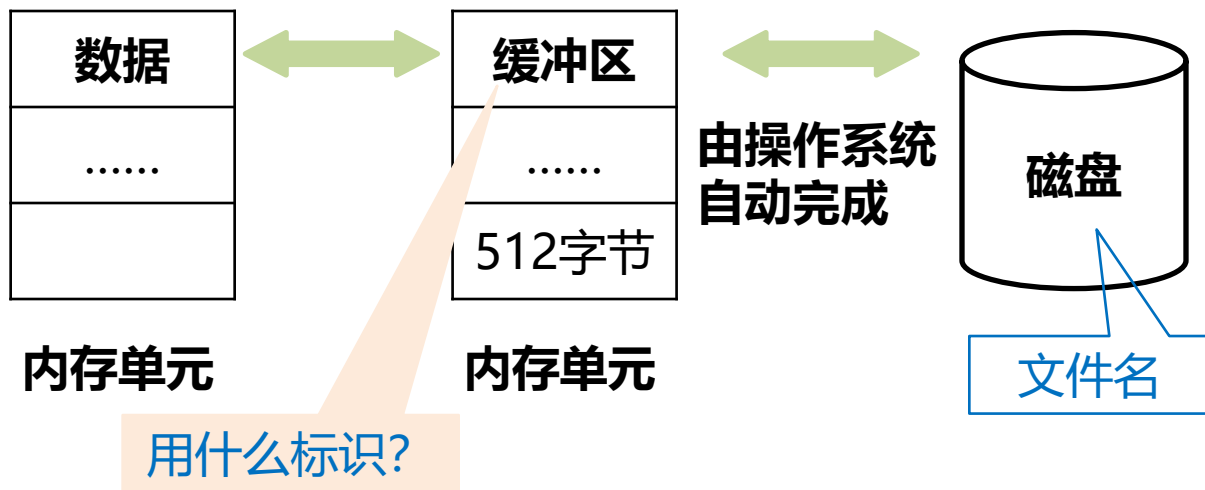
□ 磁盘访问速度与内存访问速度差别很大，直接访问磁盘效率很低





## 12.1.4 缓冲文件系统

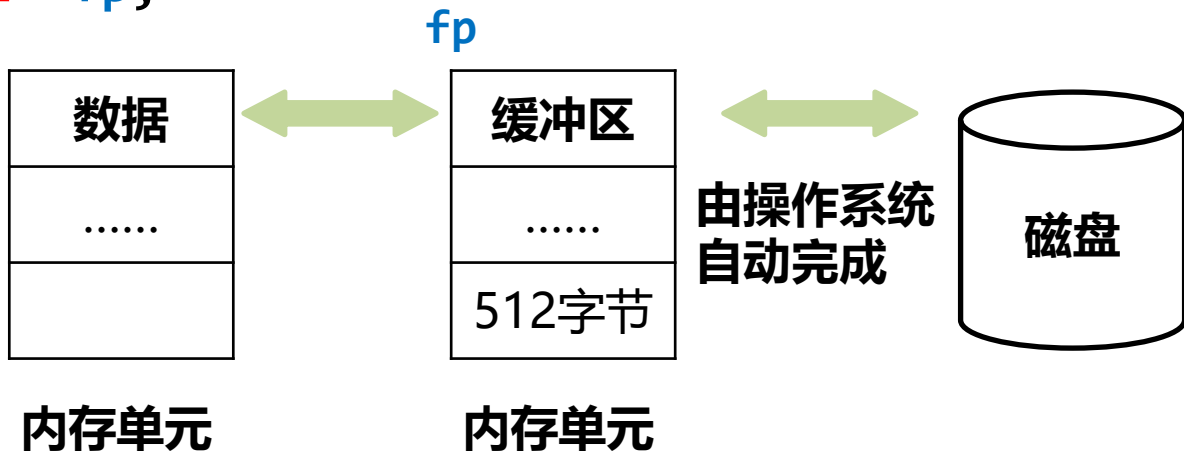
- 向磁盘输出数据：数据 ----> 缓冲区，装满缓冲区后 ----> 磁盘文件
- 从磁盘读取数据：先**一次性**从磁盘文件将**一批数据**输入到缓冲区，然后再从缓冲区**逐个**读入数据到变量



# 缓冲文件系统与文件类型指针

□ 用文件指针指示文件缓冲区中具体读写的位置

**FILE** \*fp;



□ 同时使用多个文件时，每个文件都有缓冲区，用不同的文件指针分别指示

## 12.1.5 文件结构与文件类型指针

### □ 文件结构与自定义类型typedef

FILE: 结构类型, 用typedef定义(见stdio.h)

```
typedef struct{
    short          level;          /* 缓冲区使用量 */
    unsigned       flags;          /* 文件状态标志 */
    char           fd;             /* 文件描述符 */
    short          bsize;          /* 缓冲区大小 */
    unsigned char  *buffer;        /* 文件缓冲区的首地址 */
    unsigned char  *curp;          /* 指向文件缓冲区的工作指针 */
    unsigned char  hold;           /* 其他信息 */
    unsigned       istemp;
    short          token;
} FILE;
```

## 12.1.5 文件结构与文件类型指针

### □ 自定义类型 (typedef) :

- 将C语言中的已有类型（包括已定义过的自定义类型）重新命名
- 新的名称可以代替已有数据类型
- 常用于简化对复杂数据类型定义的描述

**typedef** <已有类型名> <新类型名>;

**typedef** int **INTEGER**;

int i, j;                      <====>                      **INTEGER** i, j;

**typedef** int\* **POINTER**;

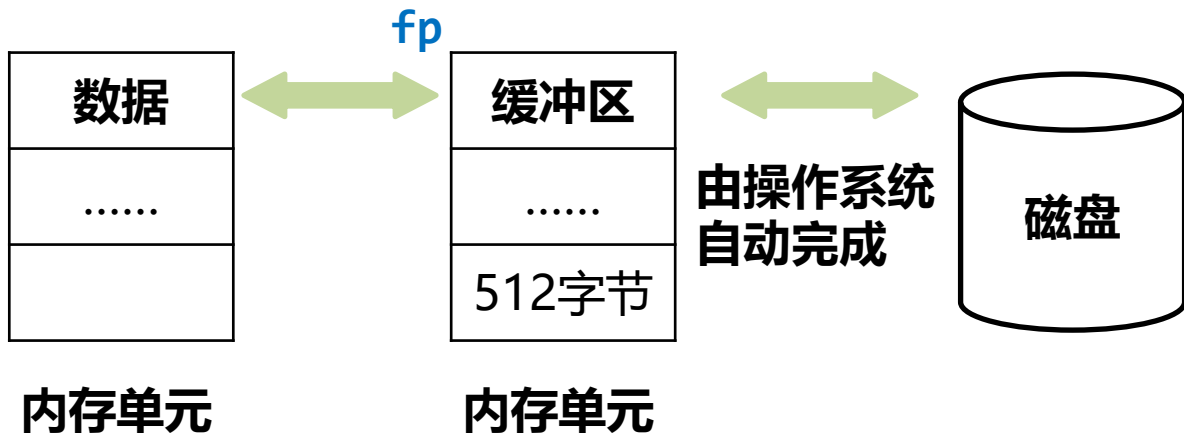
int\* p;                      <====>                      **POINTER** p;

# 文件类型指针

**FILE** \*fp;

如何使fp与具体文件挂钩?

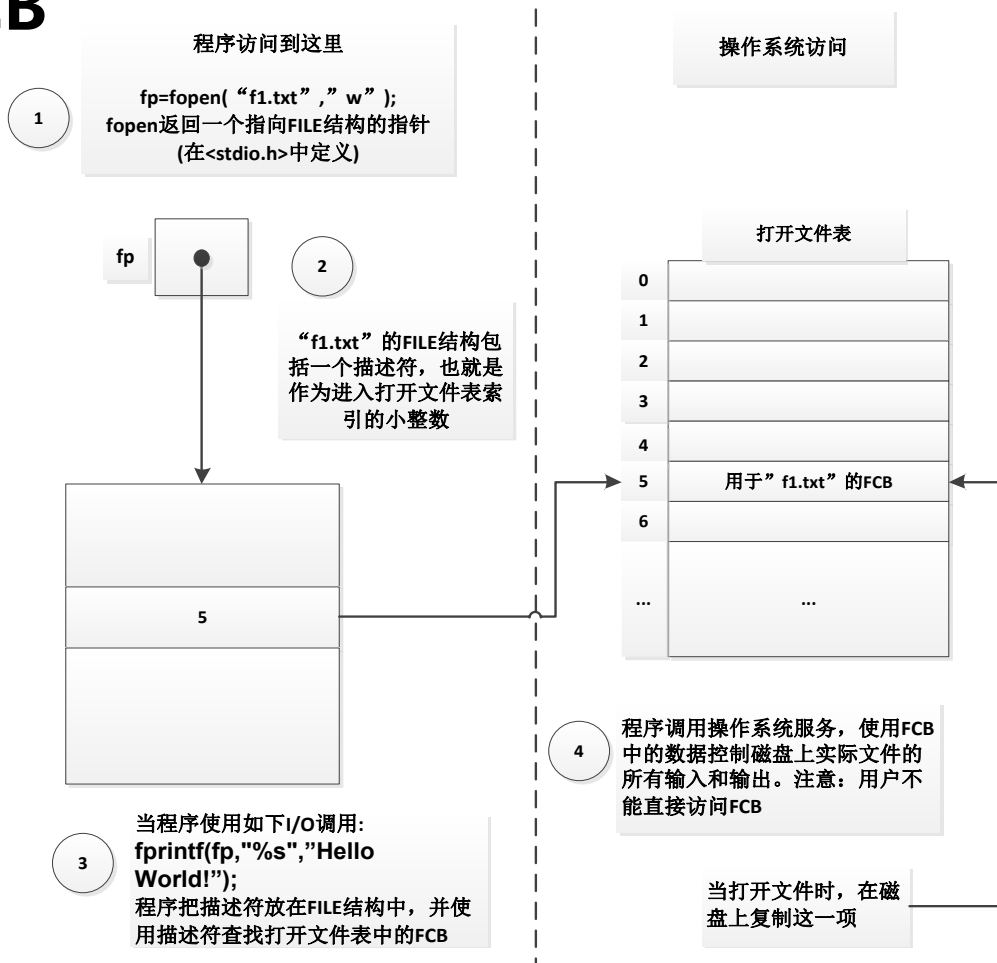
□ 指向文件缓冲区，通过移动指针实现对文件的操作



□ 同时使用多个文件时，每个文件都有缓冲区，用不同的文件指针分别指示

## \*12.1.6 文件控制块FCB

- ❑ 文件控制块FCB(File Control Block)
- ❑ OS中对文件的操作控制是通过FCB，处理的是FCB列表
- ❑ 一个文件对应一个FCB
- ❑ 文件缓冲区由程序中fopen语句动态创建
- ❑ 打开文件时，FCB的内容信息被复制到文件缓冲区保存
- ❑ 用文件指针指向文件缓冲区实现对文件数据的访问



## 12.1.7 文件处理步骤

### □ 四个步骤:

- 定义文件指针
- 打开文件: 文件指针指向磁盘文件缓冲区
- 文件处理: 文件读写操作
- 关闭文件

## 12.2 用户信息加密和校验

- 例12-2 为了保障系统安全，通常采用用户账号和密码登录系统。系统用户信息存放在一个文件中，系统账号名和密码由若干字母与数字字符构成，因安全需要文件中的密码不能是明文，必须要经过加密处理。请编程实现：输入5个用户信息（包含账号名和密码）并写入f12-2.dat。要求文件中每个用户信息占一行，账号名和加密过的密码之间用一个空格分隔。**密码加密算法**：对每个字符ASCII码的低四位求反，高四位保持不变（即将其与15进行异或）



## 12.2.1 程序解析

```
#include <stdio.h>
#include <string.h>
struct sysuser{ /*定义系统用户帐号信息结构*/
    char username[20];
    char password[8];
};
void encrypt(char *pwd);
int main(void)
{
    FILE *fp;          /*1.定义文件指针*/
    int i;
    struct sysuser su;
    if((fp=fopen("f12-2.txt","w")) == NULL){
        printf("File open error!\n");
        exit(0);
    }
    for(i=1; i<=5; i++){
        printf("Enter %d th sysuser(name password):",i);
        scanf("%s%s",su.username,su.password);
        encrypt(su.password);
        fprintf(fp,"%s %s\n",su.username,su.password);
    }
    if(fclose(fp)){
        printf("Can not close the file!\n");
        exit(0);
    }
    return 0;
}
```

```
/*加密算法*/
/*与15 (二进制码是00001111) 异或, 实现低四位取反, 高四位保持不变*/
void encrypt(char *pwd)
{
    int i;
    for(i=0; i<strlen(pwd); i++)
        pwd[i] = pwd[i] ^ 15;
}

/*2.打开文件, 进行写入操作*/

/*3. 将5位用户帐号信息写入文件*/

/*输入用户名和密码 */
/*进行加密处理*/
/*写入文件*/

/*4.关闭文件*/
```

## 12.2.2 打开文件和关闭文件

```
if((fp = fopen("f12-2.txt", "w")) == NULL){  
    printf("File open error!\n");  
    exit(0);  
}
```

### □ `fopen("文件名", "文件打开方式")`

- 使文件指针与相应文件实体对应起来
- 程序对文件指针进行操作，即fp代表磁盘文件

### □ 函数fopen()的返回值

- 执行成功，则返回包含文件缓冲区等信息的FILE型地址，赋值给文件指针fp
- 不成功，则返回一个NULL（空值）

`exit(0)`：关闭所有打开的文件，并终止程序的执行

参数0表示程序正常结束；非0参数通常表示不正常的程序结束

# 文件打开方式

```
fp = fopen("f12-2.txt", "w")
```

□ 文件打开方式参数表 read write append binary

| 文 本 文 件 (ASCII) |          | 二 进 制 文 件(Binary) |          |
|-----------------|----------|-------------------|----------|
| 使用方式            | 含 义      | 使用方式              | 含 义      |
| “ r ”           | 打开只读文件   | “ rb ”            | 打开只读文件   |
| “ w ”           | 建立只写新文件  | “ wb ”            | 建立只写新文件  |
| “ a ”           | 打开添加写文件  | “ ab ”            | 打开添加写文件  |
| “ r+ ”          | 打开读/写文件  | “ rb+ ”           | 打开读/写文件  |
| “ w+ ”          | 建立读/写新文件 | “ wb+ ”           | 建立读/写新文件 |
| “ a+ ”          | 打开读/写文件  | “ ab+ ”           | 打开读/写文件  |

# 文件读写与打开方式

## if 读文件

指定的文件必须存在，否则出错；

## if 写文件(指定的文件可以存在，也可以不存在)

if 以 "w" 方式写

if 该文件已经存在

原文件将被删去重新建立；

else

按指定的名字新建一个文件；

else if 以 "a" 方式写

if 该文件已经存在

写入的数据将被添加到指定文件原有数据的后面，不会删去原来的内容；

else

按指定的名字新建一个文件（与"w"相同）；

## if 文件同时读和写

使用 "r+"、"w+" 或 "a+" 打开文件

# 关闭文件

```
if(fclose(fp)){  
    printf("Cannot close the file!\n");  
    exit(0);  
}
```

## □ fclose(文件指针)

- 把缓冲区中的数据写入磁盘扇区，确保写文件的正常完成
- 释放文件缓冲区单元和FILE结构体，使文件指针与具体文件脱钩

## □ 函数fclose()的返回值

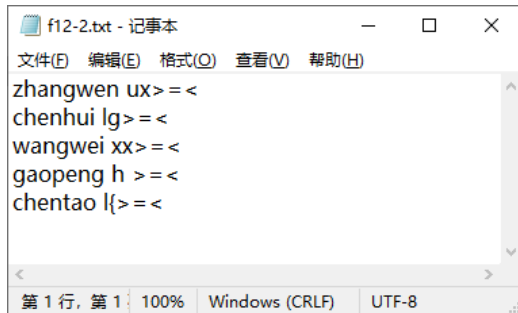
- 返回0：正常关闭文件
- 返回非0：无法正常关闭文件

## 12.2.3 文件读写

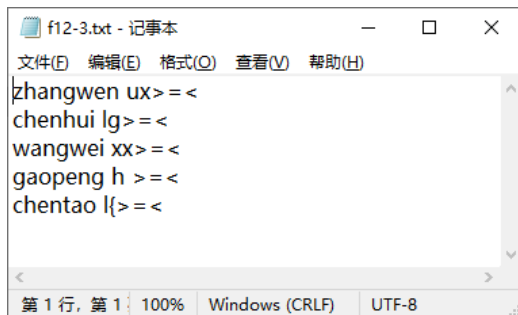
- 例12-3 复制用户信息文件。将例12-2的用户信息文件f12-2.txt文件备份一份，取名为文件f12-3.txt
- 说明：运行程序前将文件f12-2.txt与源程序放在同一目录下

## 12-3 源程序

```
#include <stdio.h>
#include <process.h>
int main(void)
{
    FILE *fp1,*fp2;
    char ch;
    if((fp1=fopen("f12-2.txt","r"))==NULL){
        printf("File open error!\n");
        exit(0);
    }
    if((fp2=fopen("f12-3.txt","w"))==NULL){
        printf("File open error!\n");
        exit(0);
    }
    while(!feof(fp1)){
        ch=fgetc(fp1);
        if(ch != EOF) fputc(ch,fp2);
    }
    if(fclose(fp1)){
        printf("Can not close the file!\n");
        exit(0);
    }
    if(fclose(fp2)){
        printf("Can not close the file!\n");
        exit(0);
    }
    return 0;
}
```



```
f12-2.txt - 记事本
文件(F)  编辑(E)  格式(O)  查看(V)  帮助(H)
zhangwen ux>=<
chenhui lg>=<
wangwei xx>=<
gaopeng h >=<
chentao l{>=<
第 1 行, 第 1 100% Windows (CRLF) UTF-8
```



```
f12-3.txt - 记事本
文件(F)  编辑(E)  格式(O)  查看(V)  帮助(H)
zhangwen ux>=<
chenhui lg>=<
wangwei xx>=<
gaopeng h >=<
chentao l{>=<
第 1 行, 第 1 100% Windows (CRLF) UTF-8
```

# 打开多个文件

```
if((fp1=fopen("f12-2.txt","r"))==NULL){  
    printf("File open error!\n");  
    exit(0);  
}  
if((fp2=fopen("f12-3.txt","w"))==NULL){  
    printf("File open error!\n");  
    exit(0);  
}
```

□ C语言允许同时打开多个文件

□ 不同的文件对应不同的文件指针

□ 不允许同一个文件在关闭前再次打开



# 文件读写函数

- 字符读写函数: `fgetc()` / `fputc()`
- 字符串读写函数: `fgets()` / `fputs()`
- 格式化读写函数: `fscanf()` / `fprintf()`
- 二进制读写函数: `fread()` / `fwrite()`
- 其它相关函数:
  - 检测文件结尾函数 `feof()`
  - 检测文件读写出错函数 `ferror()`
  - 清除末尾标志和出错标志函数 `clearerr()`
  - 文件定位的函数 `fseek()`、`rewind()`、`ftell()`

# 字符读写函数fgetc()和fputc()

## □ 函数fgetc()

□ `ch = fgetc(fp);`

从fp所指示的磁盘文件上读入一个字符到ch

□ 区分键盘字符输入函数getchar()

## □ 函数fputc()

□ `fputc(ch, fp)`

把一个字符ch写到fp所指示的磁盘文件上

□ 返回值

□ -1(EOF): 写文件失败

□ ch: 写文件成功

```
while(!feof(fp1)){  
    ch = fgetc(fp1);  
    if(ch != EOF) fputc(ch, fp2);  
}
```

# 字符串读写函数fgets()和fputs()

## □ 函数fputs()

□ `fputs(s, fp);`

向指定的文本文件写入一个字符串

□ `s`: 要写入的字符串, 结束符'`\0`'不写入文件

□ 函数返回值

□ 执行成功, 函数返回所写的最后一个字符

□ 否则, 函数返回EOF

# 字符串读写函数fgets()和fputs()

## □ 函数fgets()

□ fgets(s, n, fp);

从文本文件中读取字符串

- s: 可以是字符数组名或字符指针; n: 指定读入的字符个数; fp: 文件指针
- 函数被调用时, 最多读取n-1个字符, 并将读入的字符串存入s所指向内存地址开始的n-1个连续的内存单元中。当函数读取的字符达到指定的个数或接收到换行符, 或接收到文件结束标志EOF时, 将在读取的字符后面自动添加一个'\0'字符; 若有换行符, 则将换行符保留 (换行符在'\0'之前); 若有EOF, 则不保留
- 函数返回值
  - 执行成功, 函数返回读取的字符串
  - 否则, 则返回空指针, 这时, s的内容不确定

## 例12-4

- 例12-2的f12-2.txt文件保存着系统用户信息，编写一个函数checkUserValid()用于登录系统时校验用户的合法性。检查方法是：
  - 在程序运行时输入用户名和密码，然后在用户文件中查找该用户信息，如果用户名与密码在文件中找到，则表示用户合法，返回1，否则返回0
  - 程序运行时，输入一个用户名和密码，调用checkUserValid()函数，如果返回1，则提示"Valid user!"，否则输出"Invalid user!"
  - 提示：合法性检查的规则。由于文件中的用户名和密码按行存取，把一行看作一个字符串s1，将输入的用户名和密码加密后生成另一个字符串s2，然后通过比较s1和s2，来确定文件中是否存在用户

## 例12-4 源程序

```
/*校验用户信息的合法性, 成功返回1, 否则返回0*/
int checkUserValid(struct sysuser *psu)
{
    FILE *fp;
    char usr[30],usr1[30],pwd[10];
    int check=0;                /*检查结果变量, 初始化为0*/
    /*连接生成待校验字符串*/
    strcpy(usr,psu->username);   /*复制psu->username到usr1 */
    strcpy(pwd,psu->password);   /*复制psu->password到pwd */
    encrypt(pwd);               /*调用例12-2的encrypt对密码进行加密*/
    /*连接usr、空格、pwd和\n构成新字符串usr, 用于在文件中检查匹配*/
    strcat(usr, " "); strcat(usr,pwd); strcat(usr,"\n");
    /*打开文件"f12-2.txt"读入*/
    if((fp=fopen("f12-2.txt","r"))==NULL){
        printf("File open error!\n");
        exit(0);
    }
    /*从文件读入用户信息数据, 遍历判断是否存在*/
    while(!feof(fp)){
        fgets(usr1,30,fp);      /*读入一行用户信息字符串到usr1*/
        if(strcmp(usr,usr1)==0){ /*比较判断usr与usr1是否相同*/
            check=1; break;
        }
    }
    /*关闭文件*/
    if(fclose(fp)){
        printf("Can not close the file!\n");
        exit(0);
    }
    return check;
}
```

# 格式化文件读写fscanf()和fprintf()

## □ 指定格式的输入输出函数:

□ fscanf(文件指针, 格式字符串, 输入表);

□ fprintf(文件指针, 格式字符串, 输出表);

```
FILE *fp; int n; float x;
```

```
fp = fopen("a.txt", "r");
```

```
fscanf(fp, "%d%f", &n, &x);
```

从文件a.txt分别读入整型数到变量n、浮点数到变量x

```
fp = fopen("b.txt", "w");
```

```
fprintf(fp, "%d%f", n, x);
```

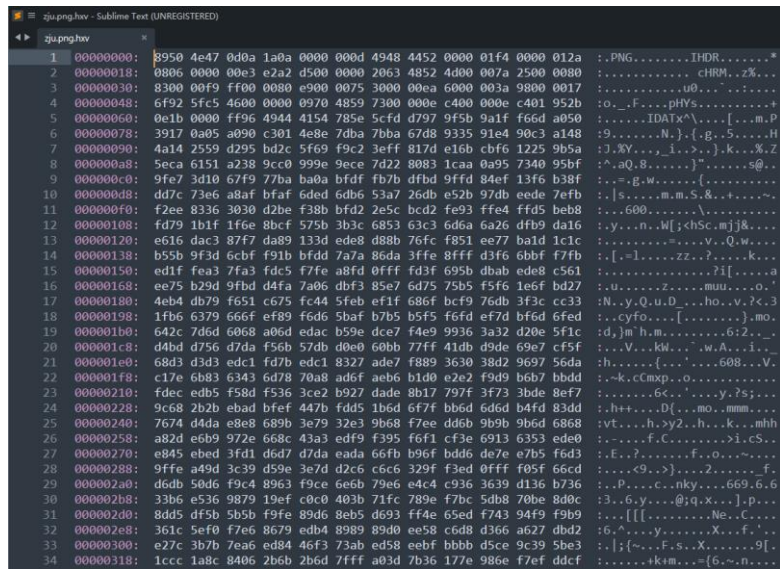
将变量n和x的数值写入文件b.txt

# 二进制文件

□ 二进制文件用文本方式显示会显示乱码，因为其本身不表示字符

□ 查看二进制字节码可以使用：

□ sublime、010 Editor等编辑器，或是编写程序输出



| Address  | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  | Dump             |
|----------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|------------------|
| 00000030 | 83 | 00 | 00 | f9 | ff | 00 | 00 | 80 | e9 | 00 | 00 | 75 | 30 | 00 | 00 | ea | ?.?..€?.u0..?□□□ |
| 00000040 | 60 | 00 | 00 | 3a | 98 | 00 | 00 | 17 | 6f | 92 | 5f | c5 | 46 | 00 | 00 | 00 | `.....o扣臚...[    |
| 00000050 | 09 | 70 | 48 | 59 | 73 | 00 | 00 | 0e | c4 | 00 | 00 | 0e | c4 | 01 | 95 | 2b | .pHYs...?..?□□□  |
| 00000060 | 0e | 1b | 00 | 00 | ff | 96 | 49 | 44 | 41 | 54 | 78 | 5e | 5c | fd | d7 | 97 | .... 眺DATx^\ ？   |
| 00000070 | 9f | 5b | 9a | 1f | 6f | 6d | a0 | 50 | 39 | 17 | 0a | 05 | a0 | 90 | c3 | 01 | 焱?鯨.P9.....?□□□  |
| 00000080 | 4e | 8e | 7d | ba | 7b | ba | 67 | d8 | 93 | 35 | 91 | a4 | 90 | c3 | a1 | 48 | N契筋纂負5或.漫HC      |
| 00000090 | 4a | 14 | 25 | 59 | d2 | 95 | bd | 2c | 5f | 69 | f9 | c2 | 3e | ff | 81 | 7d | J.¥Y視?_i > .][   |
| 000000a0 | e1 | 6b | cb | f6 | 12 | 25 | 9b | 5a | 5e | ca | 61 | 51 | a2 | 38 | 9c | c0 | 營琐.％沅+蔭Q?澀□□     |
| 000000b0 | 99 | 9e | 9e | ce | 7d | 22 | 80 | 83 | 1c | aa | 0a | 95 | 73 | 40 | 95 | bf | 橈坂)"e??眺@曠□□□    |
| 000000c0 | 9f | e7 | 3d | 10 | 67 | f9 | 77 | ba | ba | 0a | bf | df | fb | 7b | df | bd | 燴=.q鴿汉.窠麟吃□      |
| 000000d0 | 9f | fd | 84 | ef | 13 | f6 | b3 | 8f | dd | 7c | 73 | e6 | a8 | af | bf | af | 燒粿.此.軋s姉 ?[      |
| 000000e0 | 6d | ed | 6d | b6 | 53 | a7 | 26 | db | e5 | 2b | 97 | db | ee | de | 7e | fb | m鞘機?垢+棹鉅~?□□     |
| 000000f0 | f2 | ee | 83 | 36 | 30 | 30 | d2 | be | f3 | 8b | bf | d2 | 2e | 5c | bc | d2 | 蝶?00揖髻悬.\家□□     |
| 00000100 | fe | 93 | ff | e4 | ff | d5 | be | b8 | fd | 79 | 1b | 1f | 1f | 6e | 8b | cf | 鯽 ?站庚y...n增□     |
| 00000110 | 57 | 5b | 3b | 3c | 68 | 53 | 63 | c3 | 6d | 6a | 6a | 26 | df | b9 | da | 16 | W[;<hSc胛jj&Y?□□  |
| 00000120 | e6 | 16 | da | c3 | 87 | f7 | da | 89 | 13 | 3d | ed | e8 | d8 | 8b | 76 | fc | ?谈岡越.=窠獵v?□□     |
| 00000130 | f8 | 51 | ee | 77 | ba | 1d | 1c | 1c | b5 | 5b | 9f | 3d | 6c | bf | f9 | 1b | 鳴顆?..礫?1岢.□□□    |
| 00000140 | bf | dd | 7a | 7a | 86 | da | 3f | fe | 8f | ff | d3 | f6 | 6b | bf | f7 | fb | 枯zz囉?? 遇k亏?□     |
| 00000150 | ed | 1f | fe | a3 | 7f | a3 | fd | c5 | f7 | fe | a8 | fd | 0f | ff | fd | 3f | ? .} 还 ? ?□□□    |



# 数据块读写fread()和fwrite()

□ fread(buffer, size, count, fp);

从二进制文件中读入一个数据块到变量

□ fwrite(buffer, size, count, fp);

向二进制文件中写入一个数据块

□ 参数:

□ buffer: 指针, 表示存放数据的首地址

□ size: 数据块的字节数

□ count: 要读写的数据块块数

□ fp: 文件指针

□ 返回值:

□ 返回成功读取(写入)的数据块个数, 如果返回的个数与传入的count参数不同, 则可能发生了错误(对于fread函数来说也可能是到达了文件末尾)

# 例：实现copy命令

## □ 用二进制文件读写方法实现文件copy命令

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    FILE *fin, *fout;
    int rc; //read count
    unsigned char buf[BUFFER_SIZE];

    if(argc < 3){
        printf("usage: %s %s\n", argv[0], "srcfile targetfile");
        exit(1);
    }
    if((fin = fopen(argv[1], "rb")) == NULL){
        printf("failed to open source file %s\n", argv[1]);
        exit(1);
    }
    if((fout = fopen(argv[2], "wb")) == NULL){
        printf("failed to open target file %s\n", argv[2]);
        fclose(fin);
        exit(1);
    }
}
```

```
while((rc = fread(buf, sizeof(unsigned char), BUFFER_SIZE, fin)) != 0)
    fwrite(buf, sizeof(unsigned char), rc, fout);

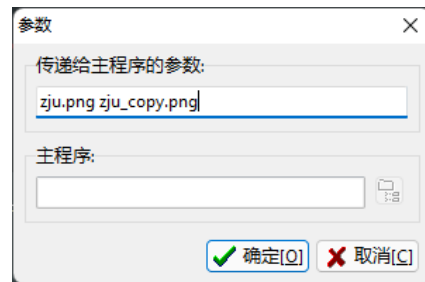
fclose(fin), fclose(fout);

return 0;
}
```

方法一：源文件编译后打开命令行窗口执行下面的命令：

```
copy.exe zju.png zju_copy.png
```

方法二：Dev C++，运行->参数，设置传递给程序的参数：



# 例：识别PNG格式文件

PNG是一种适用于网络传播的无损图片格式。其规定文件开头的8字节的PNG文件署名为0x89504e470d0a1a0a。试编写程序验证PNG文件的文件头署名。

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    unsigned char buffer[8];
    FILE *fp;

    if((fp = fopen("zju.png", "rb")) == NULL){
        printf("File open error!\n");
        exit(1);
    }

    fread(buffer, 1, 8, fp);
    for(int i = 0; i < 8; i++)
        printf("%02hhx ", buffer[i]);

    fclose(fp);
    return 0;
}
```

zju.png



|           |                     |                                         |                           |
|-----------|---------------------|-----------------------------------------|---------------------------|
| 00000000: | 8950 4e47 0d0a 1a0a | 0000 000d 4948 4452 0000 01f4 0000 012a | ::PNG.....IHDR.....*      |
| 00000018: | 0800 0000 0000 e2a2 | d500 0000 2063 4852 4d00 007a 2500 0080 | :.....CHRM..z%...         |
| 00000030: | 8300 00f9 ff00 0080 | e900 0075 3000 00ea 6000 003a 9800 0017 | :.....u0...`.....         |
| 00000048: | 6f92 5fc5 4600 0000 | 0970 4859 7300 000e c400 000e c401 952b | :o...F....pHYs.....+      |
| 00000060: | 0e1b 0000 ff96 4944 | 4154 785e 5cfd d797 9f5b 9a1f f66d a050 | :.....IDATx^\\....[...m.P |
| 00000078: | 3917 0a05 a090 c301 | 4e8e 7dba 7bba 67d8 9335 91e4 90c3 a148 | :9.....N..}{.g..5....H    |
| 00000090: | 4a14 2559 d295 bd2c | 5f69 f9c2 3eff 817d e16b cbf6 1225 9b5a | :J.%Y..._i...>..}.k...%Z  |
| 000000a8: | 5eca 6151 a238 9cc0 | 999e 9ece 7d22 8083 1caa 0a95 7340 95bf | :^..aQ.8.....}".....s@..  |
| 000000c0: | 9fe7 3d10 67f9 77ba | ba0a bfdf fb7b dfbd 9ffd 84ef 13f6 b38f | :...=.g.w.....{.....      |

## 例：读取PNG图像的宽和高

PNG图像文件将图像的宽和高记录在文件起始16字节处，宽和高分别占用4个字节。试编写程序输出某个PNG图像的宽和高。

zju.png (500 x 298)



$$0000\ 01f4 = 1 \times 16^2 + 15 \times 16 + 4 = 500$$

$$0000\ 012a = 1 \times 16^2 + 2 \times 16 + 10 = 298$$

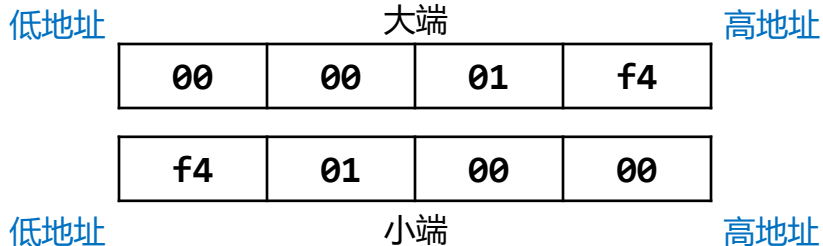
```
00000000: 8950 4e47 0d0a 1a0a 0000 000d 4948 4452 0000 01f4 0000 012a :.PNG.....IHDR.....*
00000018: 0806 0000 00e3 e2a2 d500 0000 2063 4852 4d00 007a 2500 0080 :.....cHRM..z%..
00000030: 8300 00f9 ff00 0080 e900 0075 3000 00ea 6000 003a 9800 0017 :.....u0....:....
00000048: 6f92 5fc5 4600 0000 0970 4859 7300 000e c400 000e c401 952b :o_.F....pHYs.....+
00000060: 0e1b 0000 ff96 4944 4154 785e 5cfd d797 9f5b 9a1f f66d a050 :.....IDATx^\....[...m.P
00000078: 3917 0a05 a090 c301 4e8e 7dba 7bba 67d8 9335 91e4 90c3 a148 :9.....N.}.{.g..5....H
00000090: 4a14 2559 d295 bd2c 5f69 f9c2 3eff 817d e16b cbf6 1225 9b5a :J.%Y...,_i...>..}.k...%Z
000000a8: 5eca 6151 a238 9cc0 999e 9ece 7d22 8083 1caa 0a95 7340 95bf :^..aQ..8.....}".....s@..
000000c0: 9fe7 3d10 67f9 77ba ba0a bfdf fb7b dfbd 9ffd 84ef 13f6 b38f :...=.g.w.....{.....
```

# 源程序：读取PNG图像的宽和高

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

// 大端存储转小端
uint32_t be2le(uint32_t be)
{
    return (
        ((be >> 24) & 0xff) |
        ((be >> 8) & 0xff00) |
        ((be << 8) & 0xff0000) |
        ((be << 24) & 0xff000000)
    );
}
```

对于同样的整数500，大小端模式下其字节码存储顺序不同：



```
int main()
{
    uint8_t width[4], height[4];
    FILE *fp;

    if((fp = fopen("zju.png", "rb")) == NULL){
        printf("File open error!\n");
        exit(1);
    }

    // png图像的宽和高位于文件起始16字节处,
    // 分别占4个字节
    fseek(fp, 16L, SEEK_SET);
    fread(width, 1, 4, fp);
    fread(height, 1, 4, fp);

    // png文件中存放宽和高的数据是大端的顺序
    // 需要在内存中转成小端才能在x86等小端处理器上正常打印整数
    printf("png image width: %d, height: %d",
        be2le(*(uint32_t*)width),
        be2le(*(uint32_t*)height));

    fclose(fp);

    return 0;
}
```

## 其它相关函数

### □ 函数feof()

□ `feof(fp);`

判断fp指针是否已经到达文件末尾

#### □ 返回值

□ 非0：到文件结束位置

□ 0：文件未结束

### □ 函数rewind()

□ `rewind(fp);`

定义文件指针，使文件指针指向读写文件的首地址，即打开文件时文件指针所指向的位置

## 其它相关函数

### □ 函数fseek(): 控制指针移动

□ `fseek(fp, offset, from);`

□ offset: 移动偏移量, long型

□ from: 起始位置, 文件首部、当前位置和文件尾部分别对应0, 1, 2, 或常量SEEK\_SET、SEEK\_CUR、SEEK\_END

□ 例如:

□ `fseek(fp, 20L, 0)`: 将文件位置指针移动到离文件首20字节处

□ `fseek(fp, -20L, SEEK_END)`: 将文件位置指针移动到离文件尾部前20字节处

### □ 函数ftell(): 获取当前文件指针的位置 (相对于开头的字节数)

□ `ftell(fp);`

□ 函数出错时, 返回EOF

## 其它相关函数

### □ 函数ferror()

□ `ferror(fp);`

- 检查文件在用各种输入输出函数进行读写时是否出错，若返回值为0，表示未出错，否则表示出错

### □ 函数clearerr()

□ `clearerr(fp);`

- 清空出错标志和文件结束标志，使它们为0



## 12.3 文件综合应用：个人资金账户管理

### □ 12.3.1 顺序文件和随机文件

按照C程序对文件访问的特点来分，文件可分为顺序访问文件和随机访问文件，简称为顺序文件和随机文件。前面介绍的所有例子都进行的是顺序访问，通过使用fprintf()或fputs()函数创建的数据记录长度并不是完全一致的，这种记录长度不确定的文件访问称为顺序访问。而随机访问文件要求文件中单个记录的长度固定，可直接访问，这样速度快，并且无需通过其他记录查找特定记录。因此随机文件适合银行系统、航空售票系统、销售点系统和其他需要快速访问特定数据的事务处理系统。

## 12.3.2 个人资金账户管理

### 要求

- 个人资金账户的信息统一放在随机文件中,该随机文件包括的数据项有记录ID、发生日期、发生事件、发生金额（正+的表示收入，负-表示支出）和余额。每记录一次收支，文件要增加一条记录，并计算一次余额。
- 程序实现3个功能，包括：1) 可以创建该文件并添加新收入或支出信息；2) 可以显示所有记录列表，得知资金账户的收支流水帐；3) 查询最后一条记录，获知账户最后的余额。

## cashbox.dat文件的部分内容

| LogID | CreateDate | Note        | Charge  | Balance |
|-------|------------|-------------|---------|---------|
| 1     | 2006-06-01 | alimony     | 500.00  | 500.00  |
| 2     | 2006-06-08 | shopping    | -300.00 | 200.00  |
| 3     | 2006-06-15 | shopping    | -60.00  | 140.00  |
| 4     | 2006-06-20 | workingpay  | 200.00  | 340.00  |
| 5     | 2006-08-01 | scholarship | 1000.00 | 1340.00 |
| ..... |            |             |         |         |

# 本章总结

- 文件的概念
  - 文本文件和二进制文件
  - 文件缓冲系统
  - 文件结构，文件指针，自定义类型
- 文件的打开和关闭
  - 文件处理实现过程
- 文件读写操作与常用文件操作函数
- 文件综合应用

- 掌握文件的概念，文件缓冲系统的基本原理，文件读写操作实现的基本过程；
- 能熟练使用文件进行编程；
- 掌握常用的文本文件读写操作函数；
- 了解顺序文件和随机文件的应用。

---

**谢 谢**

---